A Causality-Preserving, Stereographically Compactified Worldline Representation for Retarded Liénard–Wiechert Electrodynamics

Tim Huff
Independent Researcher (Creve Coeur, MO, USA)
Software: https://github.com/timsdsdio/hopf-hough (MIT License)

Version 6 — October 2026

Abstract

We describe and verify a method, released as the open-source tool `hopf-hough`, for evaluating the exact retarded Liénard–Wiechert (LW) field of moving point charges at every pixel of an observation plane on the GPU. Each pixel solves, for every source, the light-cone equation $f(R) = R - \|\mathbf{x} - \mathbf{z}(R)\| = 0$ on a compact per-source history, then evaluates the LW field from the interpolated history. This is a gather-side formulation, efficient when sources are few and observation points many. Three ingredients make it accurate in single-precision WebGL2: (i) the history is resampled onto shells of retarded radius $R = c(t - t_{\mathrm{ret}})$ placed by the stereographic map $R = L\tan(\theta/2)$; (ii) cubic Hermite segments are limited by a closed-form certificate on the Bernstein coefficients of $\|\mathbf{z}'(R)\|^2$, which guarantees $\|\boldsymbol{\beta}'\| \le \beta_{\max} < 1$ and hence a unique retarded root per observer; (iii) the root function is evaluated in double-float arithmetic on high/low knot positions, which lowers a single-precision accuracy floor about 100-fold (from $\approx 7\%$ to $\approx 0.05\%$ at $\gamma = 70.7$); we identify its cause. Against the exact field (no softening) at the float32-represented observer positions, with $M_T = 512$ shells, the relative L_2 error across a synchrotron pulse is 0.0095% at $\gamma = 22.4$ and 0.069% at $\gamma = 70.7$, compared with 0.22% and 7.6% for the deposited version-5 solver on the same history; at $\gamma = 224$, with double-float observer coordinates, it is 1.3% ($M_T = 512$) and 0.45% ($M_T = 2048$). A Python port of the double-precision reference agrees with the independent code PyCharge to $\sim 10^{-11}$ up to $\gamma = 7.1$. A validity indicator computed in the shader marks observers outside the stored history or where it is under-resolved; in far-field tests no unmarked observer is more than 1.6% from the exact field. No new physics is claimed.

Keywords: Liénard–Wiechert fields, retarded time, causality-preserving interpolation, Bernstein polynomials, double-float arithmetic, stereographic compactification, synchrotron radiation, WebGL2

1 Introduction and relationship to prior work

The field of a moving point charge is given in closed form by the Liénard–Wiechert expressions evaluated at the retarded time [1]. Computing it numerically at many observation points is routine in accelerator physics and in teaching, and existing tools take three routes.

1. **Grid solvers (FDTD/PIC)** such as WarpX [2] evolve $(\mathbf{E}, \mathbf{B})$ on a mesh. They handle self-consistent many-particle dynamics but suffer numerical dispersion and need spatial resolution of order $R_{\mathrm{orb}}\gamma^{-3}$ to resolve a synchrotron pulse.

2. **Wavefront and retarded-time LW codes.** Shintake's wavefront method and real-time animation [3] and LANL's CoSyR [4] construct fields from wavefronts emitted along stored trajectories. PyCharge [5] and the accompanying visualization work [6] compute exact time-domain fields from arbitrary 3D trajectories by solving the retarded-time equation per observation point, as batch computations (PyCharge 2 is JIT-compiled with JAX and can run on GPUs).

3. **Coordinate compactification** such as $R = L\tan(\theta/2)$ is standard in numerical relativity for mapping unbounded domains to finite grids [7].

Gather versus scatter. A grid solver stores the field per cell and lets sources scatter into it. The method here stores a compact history per source and lets each observation point gather: it asks each source which event on its worldline lies on the observer's past light cone. The same split between per-observer "retarded-time" and per-source "source-time-dominant" formulations is long established for moving-source noise prediction in aeroacoustics [8, 9], and per-point retarded-time solves are what PyCharge does. What this work contributes is an engineering combination: a bounded-cost, single-precision GPU kernel with a guaranteed unique root, a compact radial history, and measured, reproducible accuracy. The gather formulation is efficient when the number of sources is modest and observation points are dense; grid solvers remain the right tool for self-consistent plasmas with $\sim 10^9$ macroparticles.

Contributions

- A closed-form, branch-light causality certificate for cubic Hermite worldline segments based on the Bernstein coefficients of $\|\mathbf{z}'\|^2$ (Proposition 1). It is never stricter than the control-vector limiter of version 5 (Theorem 1). On random segment data the version-5 limiter allows a median of 0.26 of the largest blend that actually keeps $\|\mathbf{z}'\| \le \beta_{\max}$ (the new certificate: 0.46), and on smooth arcs it fires needlessly and harms accuracy.
- A diagnosis of the single-precision accuracy floor (Section 2.4): it comes from one subtraction in the root function and from storing knot positions in absolute coordinates, not from the κ^{-3} factor or Newton conditioning as version 5 stated. A double-float root function on high/low knot positions lowers it about 100-fold.
- An open, single-file browser tool and TypeScript library with a built-in verification suite, an independent cross-check against PyCharge, and a shader-side validity indicator.

2 Formulation

2.1 Retarded Liénard–Wiechert field and a cancellation-free kernel

For a point charge q with trajectory $\mathbf{z}(t_{\text{ret}})$, $\boldsymbol{\beta} = \dot{\mathbf{z}}/c$ and $\dot{\boldsymbol{\beta}} = \ddot{\mathbf{z}}/c$, let $R = c(t - t_{\text{ret}})$ be the retarded radius at observation time t, so that $d\mathbf{z}/dR = -\boldsymbol{\beta}$. The retarded radius R_{ret} for an observer at $\mathbf{x}$ solves

$$f(R) = R - \|\mathbf{x} - \mathbf{z}(R)\| = 0, \qquad \hat{\mathbf{n}} = \frac{\mathbf{x} - \mathbf{z}(R_{\text{ret}})}{\|\mathbf{x} - \mathbf{z}(R_{\text{ret}})\|}, \tag{1}$$

and the field is [1]

$$\mathbf{E}(\mathbf{x}, t) = \frac{q(1-\beta^2)}{\kappa^3 R^2}(\hat{\mathbf{n}} - \boldsymbol{\beta}) + \frac{q}{c\,\kappa^3 R}\hat{\mathbf{n}} \times \left[(\hat{\mathbf{n}} - \boldsymbol{\beta}) \times \dot{\boldsymbol{\beta}}\right], \qquad \kappa = 1 - \hat{\mathbf{n}} \cdot \boldsymbol{\beta}, \qquad \mathbf{B} = \hat{\mathbf{n}} \times \mathbf{E}. \tag{2}$$

Charges move in the x–y plane and the field is evaluated in that plane, where $\mathbf{E}$ is in-plane and $\mathbf{B}$ is along z. The tool offers an optional Plummer softening $R^2 \to R^2 + \epsilon^2$ for display near charges; all accuracy figures in this paper use $\epsilon = 0$. (Version 5 compared against a reference that applied the same softening, so its "exact" reference was not the exact field; see Section 6.)

To avoid cancellation in $1 - \hat{\mathbf{n}} \cdot \boldsymbol{\beta}$ and $1 - \beta^2$ as $\beta \to 1$, the history stores the proper velocity $\mathbf{u} = \gamma\boldsymbol{\beta}$, recovers $1 - \beta^2 = (1 + \|\mathbf{u}\|^2)^{-1}$, and evaluates

$$\kappa = \frac{1-\beta^2}{1+\beta} + \frac{1}{2}\beta\,\|\hat{\mathbf{n}} - \hat{\boldsymbol{\beta}}\|^2, \qquad \hat{\mathbf{n}} - \boldsymbol{\beta} = \frac{1-\beta^2}{1+\beta}\,\hat{\mathbf{n}} + \beta\,(\hat{\mathbf{n}} - \hat{\boldsymbol{\beta}}). \tag{3}$$

2.2 Stereographic radial shells

Each source's history is resampled onto M_T cell-centred shells $\theta_m = (m + \frac{1}{2})\Delta\theta$, $\Delta\theta = \pi/M_T$, at

$$R(\theta) = L \tan\left(\frac{\theta}{2}\right), \qquad \Delta R(\theta) = \frac{L^2 + R^2}{2L}\,\Delta\theta. \tag{4}$$

Spacing is nearly uniform for $R \lesssim L$, where radiation forms, and grows as R^2 beyond; the outermost shell is at $L\tan(\frac{\pi}{2} - \frac{\Delta\theta}{4}) \approx 4LM_T/\pi$. The map provides reach within a fixed texture, not uniform resolution to infinity: accuracy falls once ΔR no longer resolves the motion (Section 4.3). Shells are sampled from an append-only ring buffer in retarded time, so the textures are a resampled view of that buffer, refreshed each step. The buffer holds $c\,\Delta t\,N_{\text{ring}} \approx 260$ length units of history in the configuration used here; shells placed beyond that depth carry no valid history and are flagged (Section 2.6), so the usable reach is the smaller of the outermost shell and the buffer depth.

2.3 Causality certificates for cubic Hermite segments

On a segment $[R_1, R_2]$, $\Delta R = R_2 - R_1$, with knot positions $\mathbf{z}_1, \mathbf{z}_2$ and knot velocities $\boldsymbol{\beta}_1, \boldsymbol{\beta}_2$ ($\|\boldsymbol{\beta}_i\| \le \beta_{\max} < 1$), write $\mathbf{v} = (\mathbf{z}_2 - \mathbf{z}_1)/\Delta R$ and assume the chord is subluminal, $\|\mathbf{v}\| < \beta_{\max}$. A cubic Hermite curve with endpoint R-derivatives blended toward the secant by $\alpha \in [0,1]$,

$$\mathbf{V}_0(\alpha) = (1-\alpha)\mathbf{v} - \alpha\boldsymbol{\beta}_1, \qquad \mathbf{V}_2(\alpha) = (1-\alpha)\mathbf{v} - \alpha\boldsymbol{\beta}_2, \qquad \mathbf{V}_1(\alpha) = \mathbf{v} + \alpha(2\mathbf{v} + \boldsymbol{\beta}_1 + \boldsymbol{\beta}_2), \tag{5}$$

interpolates the knots for every α, and its derivative is exactly the quadratic Bernstein form $\mathbf{z}'(s) = \sum_{k=0}^{2} B_{k,2}(s)\mathbf{V}_k(\alpha)$, $s = (R - R_1)/\Delta R$.

Theorem 1 (Control-vector certificate, version 5)**.** *$\mathbf{V}_0$ and $\mathbf{V}_2$ lie in the ball $\|\cdot\| \le \beta_{\max}$ for all α by convexity. If α^* is the largest $\alpha \in [0,1]$ with $\|\mathbf{V}_1(\alpha)\| \le \beta_{\max}$ (a closed-form quadratic root), then $\|\mathbf{z}'(R)\| \le \beta_{\max}$ on the segment by the convex-hull property of Bernstein polynomials.*

The bound is a known technique: velocity limits through Bernstein control points are standard in CNC and robot trajectory planning [10, 11], and blending tangents toward the secant is a vector form of monotonicity-preserving Hermite interpolation [12]. Applied to retarded time it is useful but conservative: for a circular arc of turning angle φ, $\|\mathbf{V}_1\| = \beta(1 + \varphi^2/8 + O(\varphi^4))$, so the limiter fires on smooth arcs whose true speed never exceeds β. On the version-5 benchmark it fired on 4–87% of segments (14–87% at $M_T = 128$) that were never superluminal and caused most of the $\gamma = 70.7$, $M_T = 128$ error (Section 4).

Proposition 1 (Squared-speed certificate)**.** $\|\mathbf{z}'(s)\|^2 = \sum_{k=0}^{4} B_{k,4}(s)\,c_k$ *with*

$$c_0 = \mathbf{V}_0\cdot\mathbf{V}_0, \quad c_1 = \mathbf{V}_0\cdot\mathbf{V}_1, \quad c_2 = \tfrac{1}{3}\left(2\,\mathbf{V}_1\cdot\mathbf{V}_1 + \mathbf{V}_0\cdot\mathbf{V}_2\right), \quad c_3 = \mathbf{V}_1\cdot\mathbf{V}_2, \quad c_4 = \mathbf{V}_2\cdot\mathbf{V}_2.$$

Each $c_k(\alpha)$ is quadratic in α, and $c_k(0) = \|\mathbf{v}\|^2 < \beta_{\max}^2$. Let α_{sq} be the smallest $\alpha \in (0,1]$ at which some $c_k(\alpha) = \beta_{\max}^2$ (or 1 if none); if $\|\mathbf{v}\| = \beta_{\max}$, set $\alpha_{sq} = 0$. Then for every $\alpha \le \alpha_{sq}$, $\|\mathbf{z}'\| \le \beta_{\max}$ on the segment, and $\alpha_{sq} \ge \alpha^*$.

Proof. The product of two quadratic Bernstein expansions is a quartic Bernstein expansion with the stated coefficients; since $B_{k,4} \ge 0$ and $\sum_k B_{k,4} = 1$, $\|\mathbf{z}'\|^2 \le \max_k c_k \le \beta_{\max}^2$. c_0 and c_4 never exceed $\beta_{\max}^2$ by Theorem 1, and by continuity no c_k exceeds it before its first crossing. If all $\|\mathbf{V}_k\| \le \beta_{\max}$ then by Cauchy–Schwarz every $c_k \le \beta_{\max}^2$, so no c_k exceeds $\beta_{\max}^2$ on $[0, \alpha^*]$; outside degenerate tangential contact this gives $\alpha_{\text{sq}} \ge \alpha^*$. □

Computing α_{sq} costs three quadratic solves. The guarantee holds in exact arithmetic; in a float32 emulation it held for $\beta \le 0.99999$ with $\beta_{\max} = 0.9999995$ but was violated by up to $\sim 1 - \beta_{\max}$ at $\beta = 0.999999$, so $\beta_{\max} - \beta$ must exceed a few float32 ulps. With the speed bound on every segment, $f'(R) = 1 + \hat{\mathbf{n}}\cdot\mathbf{z}'(R) \ge 1 - \beta_{\max} > 0$, so f is strictly increasing on the stored domain

and each observer has exactly one retarded root there. The limited worldline is C^0 everywhere and C^1 at a knot when neither neighbouring segment is limited; where a limiter acts, the velocity direction can be discontinuous at the knot. A globally C^1 subluminal cubic does not always exist (for alternating chords $\pm c$ with free knot slopes it requires $c \le 2\beta/3$), whereas raising the degree restores it; we leave that for future work.

2.4 Single-precision accuracy and its cause

We attributed the error of the version-5 float32 algorithm, in a CPU emulation, by promoting one stage at a time to float64 [16]. At $\gamma = 70.7$ ($M_T = 512$), rounding only the root-function arithmetic $\mathbf{x}-\mathbf{z}$, $\|\cdot\|$ and $R-\|\cdot\|$ to float32 produces 12.4% error; storing knot positions in absolute float32 coordinates produces 2.4%; every other stage (κ and the LW kernel, Newton conditioning, the velocity and acceleration interpolation, shell radii) contributes at most 0.05%. The cause is conditioning: near the pulse crest $f'(R) \approx \kappa \sim \gamma^{-2}$, so an absolute error δ in f moves the retarded point by δ/κ, and float32 positions near $|\mathbf{x}| \sim 1$ carry $\delta \sim 6 \times 10^{-8}$.

The fix is local. Knot positions are stored as pairs $\mathbf{z} = \mathbf{z}_{\mathrm{hi}} + \mathbf{z}_{\mathrm{lo}}$ of float32 values, chords are formed from both parts, and $f(R)$ is evaluated with error-free transformations (double-float arithmetic [13, 14]); everything else stays float32. Error-free transformations only work if the compiler evaluates them as written. Apple's Metal shader compiler (reached through ANGLE) does not: on an M1 Max, an earlier version that guarded some intermediates by multiplying by a uniform equal to 1 fell back to float32 accuracy, because unguarded sums such as $(a-x)+(b-y)$ were regrouped as $(a+b)-(x+y)$. Guarding with the uniform on every operation was still not enough. The shader therefore routes every rounding step through an identity the compiler cannot see through (reinterpreting the float as an integer and XOR-ing it with a uniform equal to 0) and splits operands for the exact products by masking mantissa bits. A probe page in the repository tests each building block on the user's GPU against exact results.

Two further float32 effects appear above $\gamma \approx 100$ and were corrected in this version. (i) The chord-speed clamp that guards against superluminal input compared float32 differences of adjacent knots; at $M_T = 2048$ their relative rounding ($\sim 3 \times 10^{-5}$) exceeds $1-\beta$, so physical knots were wrongly moved. The clamp now uses the full high/low positions in double precision. (ii) The Newton derivative was floored at 5×10^{-5}, larger than the true f' at $\gamma = 224$; the floor is now $\frac{1}{2}(1-\beta_{\mathrm{max}})$, which the certificate guarantees (in exact arithmetic) is below f'.

How much accuracy is available. At $\gamma = 70.7$ the pulse crossed by the benchmark sweep is about 15 float32 ulps wide in observer position, and moving an observer by one ulp changes $\mathbf{E}$ by up to 71% of its rms value. Rounding the observer coordinates alone shifts the exact field by 3.2% (Table 1). A float32-addressed method can therefore only be judged against the exact field at the rounded point, which is the metric used here, or by placing observers with double-float coordinates, which the library's readback API supports (the rendered view uses float32 pixel coordinates) and which is required above $\gamma \approx 100$.

2.5 Root search

Because f is monotone at the stored knots (subluminal chords), the solver first bisects over knot indices, evaluating f at knots only (three texel fetches each, no spline evaluation), and then runs safeguarded Newton inside the bracketing segment on the certified cubic, iterating to convergence (at most 12 steps, with bisection fallback). The segment index and the fraction within it are kept as separate values, so float32 resolution of a combined coordinate never limits the retarded point.

2.6 Validity indicator

For each source the shader reports $q = \max(\Delta R\,\gamma\,|\dot{\beta}_\perp|/(c\beta^2),\ 6\,\|\Delta\mathbf{u}\|/(\bar{u}+0.1))$ at the retarded point: shell spacing times γ over the local radius of curvature $\rho = c\beta^2/|\dot{\beta}_\perp|$, and the relative change of proper velocity across the segment. It reports $q = -1$ when the retarded point lies beyond the outermost shell or the segment reaches past the ring-buffer depth, and raises q inside the first shell and on shells damped by the optional Nyquist taper. The app hatches pixels with $q > 1.5$ or $q < 0$. The thresholds are heuristic; Section 4.3 measures how well they predict the error.

3 Verification protocol

Tables 1–3, the straight-line and targeted limiter tests and the cost figures come from `tests/gpu/tables.mjs` (raw data in `results/tables.json`); the error attribution, the limiter statistics, the far-field estimates and the preset defect come from the scripts and saved outputs in `validation/analysis/`; the PyCharge comparison from `validation/pycharge/`.

1. **Reference.** The exact field is computed in float64 directly on the analytic trajectory (32-step bracketed bisection on t_{ret}, then up to 12 safeguarded Newton steps) with $\epsilon = 0$. With softening off, a Python port of this reference agrees with PyCharge 2.0.1 [5], which obtains velocity and acceleration by automatic differentiation of the position, to at most 1.3×10^{-11} relative error for circular orbits ($\beta = 0.5, 0.9, 0.99$, including pulse sweeps), harmonic oscillation and straight-line motion; the TypeScript reference reproduces the version-5 float64 values exactly. The reference is not externally checked above $\gamma = 7.1$.

2. **Benchmark.** A charge on a circular orbit of radius 0.85 ($c = 2.35$). A ring buffer ($\Delta t = 0.00675$) is seeded and advanced 16 steps; the shells are populated from it. Observers: 256 points at fixed distance from the orbit centre crossing the pulse emitted tangentially at retarded radius 1.15 (half-width $\max(10^{-6}, 4\gamma^{-3})$ rad), and a 64-point ring at radius 1.15.

3. **Readback.** Fields are read back from an `RGBA32F` framebuffer (`gl.readPixels`). The tables use WebGL2 with the SwiftShader software renderer, which implements IEEE float32. The in-app suite on an Apple M1 Max (Metal) reproduces Table 1 to within run-to-run scatter (e.g. 0.0089% and 0.059% at $\gamma = 22.4$ and 70.7, $M_T = 512$; `results/suite_m1max.tsv`); other GPUs should be checked with the suite.

4. **Regression.** The single-file solver deposited with version 5 is kept in the repository. The TypeScript port reproduces all 154 values of its verification suite exactly in the same browser engine; the version-5 column in Table 1 is that deposited shader run on the same history, against the unsoftened reference.

4 Results

4.1 Accuracy across Lorentz factors

Relative to the version-5 solver, the error falls 110–150$\times$ at $\gamma = 70.7$, 23–43$\times$ at $\gamma = 22.4$ for $M_T \geq 512$, and 2$\times$ at $\gamma = 22.4$, $M_T = 128$, where interpolation of the coarse history dominates. The largest change is at $M_T = 128$, $\gamma = 70.7$ (48% to 0.37%), where the version-5 limiter was active across the pulse. At $\gamma \leq 7$ both algorithms are accurate; the remaining error there is interpolation of the history and converges with M_T.

β	γ	M_T	**this version**	**this version, df64 observers**	version-5 algorithm	ring (this version)	observer rounding alone
0.8	1.67	128	0.0067%	0.0067%	0.0067%	0.0030%	2.2×10^{-5}%
0.8	1.67	512	4.5×10^{-4}%	4.4×10^{-4}%	0.001%	1.5×10^{-4}%	
0.8	1.67	2048	4.5×10^{-5}%	4.6×10^{-5}%	0.0034%	2.8×10^{-5}%	
0.95	3.20	128	0.017%	0.017%	0.017%	0.12%	3.1×10^{-4}%
0.95	3.20	512	0.0010%	0.0010%	0.0019%	0.0063%	
0.95	3.20	2048	1.5×10^{-4}%	1.3×10^{-4}%	0.0075%	3.8×10^{-4}%	
0.99	7.09	128	0.038%	0.038%	0.039%	0.012%	0.0034%
0.99	7.09	512	0.0023%	0.0023%	0.0097%	8.6×10^{-4}%	
0.99	7.09	2048	4.3×10^{-4}%	6.0×10^{-4}%	0.015%	8.1×10^{-5}%	
0.999	22.4	128	0.13%	0.13%	0.26%	0.013%	0.11%
0.999	22.4	512	**0.0095%**	0.0096%	0.22%	0.0016%	
0.999	22.4	2048	0.0067%	0.0055%	0.29%	3.3×10^{-4}%	
0.9999	70.7	128	0.37%	0.43%	48%	0.045%	3.2%
0.9999	70.7	512	**0.069%**	0.069%	7.6%	0.0027%	
0.9999	70.7	2048	0.051%	0.043%	7.8%	1.7×10^{-4}%	
0.99999	224	128	—	7.0%	—	0.041%	> 100%
0.99999	224	512	—	**1.3%**	—	9.6×10^{-4}%	
0.99999	224	2048	—	**0.45%**	—	1.2×10^{-4}%	

Table 1: Relative L_2 error of GPU readback against the exact LW field (no softening). Sweep: 256 observers crossing the synchrotron pulse; ring: 64 observers at radius 1.15. "This version" rounds observers to float32 and compares with the exact field at the rounded point; "df64 observers" places them with double-float coordinates. At $\gamma = 224$ float32 observer coordinates cannot resolve the pulse (last column), so only the df64 column is meaningful. The version-5 algorithm uses a float32 root function, the Theorem 1 limiter and bisection on the spline; it was not designed for $\gamma > 100$.

128 shells, $R \in [0, 220]$	sweep	ring	spacing at $R = 1.15$	first spacing
Stereographic $L \tan(\theta/2)$, $L = 1.35$	**0.038%**	0.012%	0.029	0.017
Logarithmic $r_0(e^{\alpha u} - 1)$	0.33%	0.080%	0.081	0.0035
Quadratic $R_{\max} u^2$	1.6%	0.69%	0.27	0.027
Uniform	> 100%	> 100%	1.72	1.72

Table 2: Same memory and reach, $\beta = 0.99$, prefilter off for every grid (version 5 applied it only to the stereographic grid, and its quadratic-grid entry of 0.21% does not reproduce). The stereographic map's advantage here follows from Eq. (4): near-uniform spacing across the region $R \lesssim L$ that contains the orbit and the observation radius, where the tested logarithmic grid concentrates shells near $R = 0$. Other parameter choices for the competing grids may narrow the gap.

M_T (outermost shell)	$R = 1.15$	$R = 3$	$R = 10$	$R = 30$	$R = 100$	$R = 200$
128 (220)	0.0064% (0)	0.61% (0)	2.9% (64)	> 100% (64)	> 100% (64)	> 100% (64)
512 (880)	0.0015% (0)	0.011% (0)	0.20% (26)	31% (64)	> 100% (64)	> 100% (64)
2048 (3520)	9.2×10^{-5}% (0)	0.0019% (0)	0.010% (0)	0.18% (64)	> 100% (64)	> 100% (64)

Table 3: Circular orbit, $\beta = 0.9$: relative L_2 error on rings of radius R around the orbit centre, and in parentheses the number of the 64 observers marked by the validity indicator. The first column gives the outermost shell; the ring buffer holds 260 length units of history. Across all far-field cases run ($\beta = 0.5$ and 0.9; $M_T = 128$, 512, 2048; taper on and off), the L_2 error over unmarked observers on a ring is at most 0.61%, and the worst single unmarked observer is 1.6% (at the edge of a marked region).

4.2 Radial grid at equal memory

4.3 Far field and the validity indicator

Accuracy falls with distance once the shell spacing, which grows as R^2, no longer resolves the orbital motion; the stored history reaches far beyond the radius at which it stays accurate. The indicator is conservative (it marks some observers whose error is below 1%). For unaccelerated motion its two curvature terms vanish, so the ring-buffer test is what protects it: with $\beta = 0.5$ straight-line motion and observers 50–800 units away, every observer with a retarded point beyond the buffer is marked (errors 2–69%) and every unmarked one is accurate to 3×10^{-5}%. An earlier draft lacked this test and left such errors unmarked. The optional Nyquist taper of version 5 damps the radiation term on coarse shells; it is off by default, and the shells it modifies are marked.

4.4 When the causality limiter matters

On smooth histories resampled onto the shells, the unlimited cubic rarely exceeds c. In the synchrotron benchmark its peak speed equals the knot speed. For hyperbolic motion (constant proper acceleration over 100 time units, $\gamma \approx 11$ at the observation time) it reaches $\|\mathbf{z}'\| = 1.057$, but only on the outermost shell ($R \in [73, 220]$, $M_T = 128$). In two such runs (proper acceleration 0.25 and 0.15), 256 observers placed so that their retarded points fall in that segment see 73% and 22% error with the squared-speed certificate, 96% and 49% with Theorem 1, and 93% and 68% with no limiter; the indicator marks all of them. On under-resolved segments a limiter can also raise the error: in the version-5 float64 analysis the squared-speed certificate gave 296% against 70% without a limiter ($\gamma = 22.4$, $M_T = 128$, $R = 8$). The certificate's role is therefore a guarantee, one root per observer so the search cannot take a wrong branch, at no cost on resolved histories; it does not make an under-resolved history accurate.

4.5 Cost

In software rendering (640 $\times$ 400 pixels, 16 sources, $M_T = 512$, three repetitions) the production kernel took 1.1–1.8$\times$ the time of the version-5 algorithm, with large run-to-run variation; an operation count estimates about 2.7$\times$ more float32 arithmetic per root-function evaluation for the double-float version. Software timings say little about GPU cost. On an Apple M1 Max GPU, a 256×256 grid takes 5–7 ms for one source and 20 ms for 16 sources at $M_T = 2048$, including upload and readback; each additional source costs about 1 ms per 65,536 observers (`validation/benchmarks/results_webgl_m1max.json`).

4.6 Comparison with PyCharge

To separate the algorithm from the language, we ported the method to JAX (float64) and ran it against PyCharge 2.0.1 [5] on the same CPU, the same problems (charges on circular orbits of radius 0.85) and the same 256×256 grid, against the exact field (`validation/benchmarks/`). PyCharge was run both with its default solver and with the cheapest fixed-point/Newton budgets that converge on the full grid.

The two are tied for $\gamma \lesssim 7$ and for 16 sources; at $\gamma = 22$ and 71 the method is 5$\times$ and 9$\times$ faster, because PyCharge's fixed-point iteration needs more steps as $\beta \to 1$ while the knot search and Newton refinement here have a cost independent of γ. The trade is accuracy: a few times 10^{-5} instead of machine precision. The second difference is precision. PyCharge requires float64, which consumer GPUs run at a small fraction of float32 speed and Apple GPUs do not support. Forced to float32, PyCharge's error on the pulse sweep is 0.027%, 0.86% and 27% at $\gamma = 7.1$, 22.4 and 70.7, against 0.0023%, 0.0095% and 0.069% for the float32 WebGL kernel with its double-float root function ($M_T = 512$). PyCharge also handles 3D and self-consistent dynamics, which

case (65,536 points)	PyCharge, default	PyCharge, cheapest converged	this method (JAX), $M_T = 2048$	its error
$\beta = 0.9$, 1 source	0.27 s	0.040 s	0.038 s	1.1×10^{-6}
$\beta = 0.99$, 1 source	0.28 s	0.066 s	0.061 s	6.2×10^{-6}
$\beta = 0.999$, 1 source	0.53 s	0.18 s	0.034 s	2.4×10^{-5}
$\beta = 0.9999$, 1 source	0.33 s	0.34 s	0.039 s	2.9×10^{-5}
$\beta = 0.9$, 16 sources	5.1 s	0.61 s	0.66 s	1.0×10^{-6}

Table 4: Float64 on a 2-core CPU, medians of three runs after compilation. PyCharge is exact when it converges (10^{-11}–10^{-14}); the method here interpolates a sampled history.

this method does not; this method works from any sampled history rather than a differentiable analytic trajectory.

5 Limitations and scope

1. **Two dimensions and prescribed motion.** Charges move in a plane and do not respond to each other's fields; there is no radiation reaction.

2. **History, not far-field storage.** The shell textures are a resampled view of an append-only ring buffer in retarded time, refreshed every step. Coarsening old history removes the radiation pulse, which needs full-resolution acceleration. Handing off to a stored far field on null slices $(\tau, \hat{\mathbf{n}})$ would only be valid beyond $R \gtrsim 1.75\,\gamma\rho/\varepsilon$ for relative error ε (ρ the radius of curvature; $175\,\gamma\rho$ at 1%), and only after removing the Fresnel arrival delay, which requires each source's emission point; without that correction the 1% radius is $\approx 210\,\rho\gamma^3$ and would cost more memory than the worldline, because the far field undoes the γ^3 compression of the emission time; the worldline is close to the most compact representation. (Version 5 proposed such a hand-off at $R \approx R_{\text{Nyq}}$ and a Hopf-bundle representation of the polarization; both are withdrawn. A transverse field over the sphere needs no bundle when stored as a Cartesian vector orthogonal to $\hat{\mathbf{n}}$, and this pattern is the established Cauchy-characteristic extraction [15].)

3. **Very high γ.** The method is tested to $\gamma = 224$. Above $\gamma \approx 100$ observers must be placed with double-float coordinates, which is available through the readback API but not in the rendered view.

4. **Hardware.** Accuracy was measured with an IEEE float32 software renderer and on one GPU (Apple M1 Max). Other compilers may transform floating-point code in ways the barrier does not anticipate; the probe page and the in-app suite check this on the user's GPU.

5. **Sources faster than the wave speed.** In media or acoustics, sources can outrun the wave and observers see several retarded roots. The certificate does not apply there; counting all roots per segment is a natural extension.

6 Changes from version 5

- The "exact" reference in version 5 applied the Plummer softening ($\epsilon = 0.06$). All errors here are against the unsoftened field.

- The single-precision floor was attributed in version 5 to κ^{-3} evaluation and Newton conditioning; the measured cause is the root-function subtraction and absolute knot storage (Section 2.4). The 6–9% GPU-versus-CPU discrepancy reported there is of the same size as the change produced by legal reorderings of float32 operations (5.7–7.3%), so it need not be a hardware effect.

- The Theorem 1 limiter caused most of the $\gamma = 70.7$, $M_T = 128$ error; it is replaced by Proposition 1. Version 5's Float64 convergence column ran without any limiter, so it measured a different algorithm from the GPU path.
- Version 5's far-field result (1.62% at $R_{\text{obs}} = 45$) used a preset that moved at $0.44c$ with no acceleration and jumped back every 4.4 time units; it tested a Coulomb-like field, not radiation. It is replaced by Table 3.
- Some version-5 table entries do not reproduce with its own code: the quadratic grid (4.4% measured vs 0.21% printed) and the $\beta = 0.999$, $M_T = 128$ cell (0.25% vs 2.47%); the max $|\Delta \mathbf{z}|$ column does not match the deposited suite's emission-point error (about 10^{-7}); and two far-field cells (quadratic and logarithmic grids) were not computed by the deposited suite.
- The ensemble ("swarm") preset double-counted the frame rotation in its acceleration (4–15% RMS error in $\dot{\boldsymbol{\beta}}$, depending on the source) and rescaled velocity without position; both are fixed.
- Prior work on aeroacoustic retarded-time formulations, velocity-bounded Bézier curves and PyCharge is now cited; the far-field hand-off and Hopf-bundle proposal are withdrawn (Section 5).

7 Code and data availability

The tool, library, tests and the scripts that produce every number in this paper are at `https://github.com/timsdsdio/hopf-hough` under the MIT License, together with the version-5 solver, the PyCharge cross-check and the raw results (`results/tables.json`). The app runs from a single HTML file with no installation.

Acknowledgments and AI assistance

The physical idea — replacing a global grid with per-source compactified histories, inspired by parameter-space (Hough) transforms — and the direction of this work are the author's. AI systems contributed substantially, and all results were produced by code in the repository and checked by the author:

- **Gemini Argon (Google)**: the original WebGL2 shader, the Theorem 1 construction, the cancellation-free kernel, the version-5 verification suite and drafts of earlier versions.
- **Astra (OpenAI)**: adversarial review of earlier drafts and code (six rounds), including the pulse-crossing sweep and the matched-reach grid benchmark.
- **Claude (Anthropic), Claude Fable 5.1 and Claude Opus 5.5**: review of version 5; a multi-agent analysis that identified the limiter's conservatism and the squared-speed certificate, the cause of the single-precision floor and the double-float fix, the far-field and prior-art issues, and the preset defects; the TypeScript port, production shader, tests, PyCharge cross-check and documentation; and drafting of this version.

References

[1] J. D. Jackson, *Classical Electrodynamics*, 3rd ed. (Wiley, 1999), Ch. 14.

[2] J.-L. Vay *et al.*, "Warp-X: A new exascale computing platform for beam–plasma simulations," *Nucl. Instrum. Methods Phys. Res. A* **909**, 476–479 (2018).

[3] T. Shintake, “New mathematical method for radiation field of moving charge,” in *Proc. EPAC 2002*, Paris, pp. 1667–1669 (2002).

[4] C.-K. Huang *et al.*, “CoSyR: a novel beam dynamics code for the modeling of synchrotron radiation effects,” arXiv:2109.14901 (2021), published in *Nucl. Instrum. Methods Phys. Res. A*; code at https://github.com/lanl/cosyr.

[5] M. Filipovich and S. Hughes, “PyCharge: An open-source Python package for self-consistent electrodynamics simulations of Lorentz oscillators and moving point charges,” *Comput. Phys. Commun.* **274**, 108291 (2022).

[6] M. Filipovich and S. Hughes, “Space-time computation and visualization of the electromagnetic fields and potentials generated by moving point charges,” *Am. J. Phys.* **89**, 482–489 (2021).

[7] A. Zenginoğlu, “Hyperboloidal foliations and scri-fixing,” *Class. Quantum Grav.* **25**, 145002 (2008).

[8] K. S. Brentner, “Numerical algorithms for acoustic integrals — the devil is in the details,” AIAA Paper 96-1706 (1996).

[9] K. S. Brentner, “An efficient and robust method for predicting helicopter high-speed impulsive noise,” *J. Sound Vib.* **203**, 87–100 (1997).

[10] R. T. Farouki, *Pythagorean-Hodograph Curves: Algebra and Geometry Inseparable* (Springer, 2008).

[11] J. Tordesillas and J. P. How, “MINVO basis: Finding simplexes with minimum volume enclosing polynomial curves,” arXiv:2010.10726 (2020).

[12] F. N. Fritsch and R. E. Carlson, “Monotone piecewise cubic interpolation,” *SIAM J. Numer. Anal.* **17**, 238–246 (1980).

[13] A. Thall, “Extended-precision floating-point numbers for GPU computation,” ACM SIGGRAPH Research Posters (2006).

[14] G. Da Graça and D. Defour, “Implementation of float-float operators on graphics hardware,” arXiv:cs/0603115 (2006).

[15] N. T. Bishop, R. Gómez, L. Lehner, M. Maharaj and J. Winicour, “High-powered gravitational news,” *Phys. Rev. D* **56**, 6298 (1997).

[16] Analysis scripts, saved outputs and reports for the error attribution, the limiter study and the far-field estimates: `validation/analysis/` in the software repository.